



ALGORITMOS Y ESTRUCTURAS DE DATOS

Apunte de Teoría

UNIDAD I: Introducción a la Algoritmia
Parte B: Estructuras de Control

CICLO LECTIVO 2018

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Tabla de contenido

OBJETIVOS	3
CONTENIDOS	3
ESTRUCTURAS DE CONTROL	4
Estructura Secuencial	4
Estructura Condicional	6
Estructuras Repetitivas.....	10

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

OBJETIVOS

- a. Identificar las acciones involucradas en un problema.
- b. Reconocer los estados de los objetos antes y después de la acción realizada.
- c. Descomponer una acción en términos de otras más elementales.
- d. Utilizar un léxico mínimo común.
- e. Comprender el significado del término Algoritmo

CONTENIDOS

Estructuras de Control:

- Secuenciales
- Condicionales
- Repetitivas

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

ESTRUCTURAS DE CONTROL

Si recordamos *la definición y características* que debe poseer un algoritmo escrito en pseudocódigo; sabremos que lo que estamos haciendo al utilizar pseudocódigo es lo que llamamos **programación estructurada**.

Todo **Algoritmo estructurado** puede ser escrito utilizando solamente tres tipos de estructura de control de secuencia:

- a) secuenciales
- b) selectivas o de decisión
- c) repetitivas

Las **estructuras de control** tienen una finalidad bastante definida:
señalar orden en que tienen que sucederse los pasos de un algoritmo.

Estructura Secuencial



La **estructura secuencial** es aquella en la que una acción (instrucción) sigue a otra en secuencia. Las tareas se suceden de tal modo que la salida de una es la entrada de la siguiente y así sucesivamente hasta el fin del proceso.

A continuación dejamos disponible un ejemplo para que lo analices:

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

EJEMPLO 1

Mostrar por pantalla el doble de un número ingresado por el usuario

ACCION Ejemplo1 ES

AMBIENTE

a,doble: entero

PROCESO

ESCRIBIR("Ingrese el número")

LEER(a)

doble := a * 2

ESCRIBIR("Resultado →",doble)

FIN_ACCION

Los algoritmos definidos hasta este punto **se ejecutan de modo secuencial**, es decir, una sentencia después de otra. La ejecución comienza con la primera sentencia del algoritmo y prosigue hasta la última sentencia, cada una de las cuales se ejecuta una sola vez. Esta forma de programación es adecuada para programas sencillos.

Sin embargo, para la resolución de problemas de tipo general se necesita la capacidad de **controlar cuáles son las sentencias que se ejecutan, y en qué momentos**.

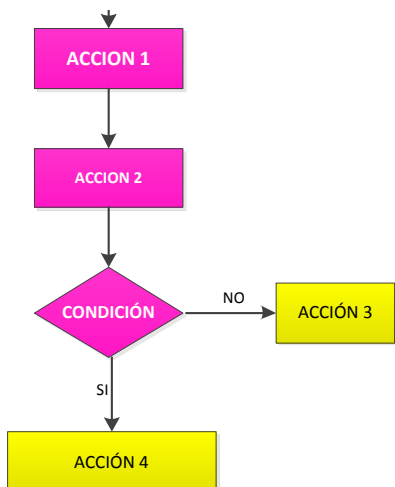


El hecho que las acciones se ejecuten secuencialmente implica que nunca se ejecutara más de una acción al mismo tiempo.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Estructura Condicional



No todos los problemas pueden resolverse empleando estructuras secuenciales. Cuando hay que tomar una decisión aparecen las estructuras condicionales. En nuestra vida diaria se nos presentan situaciones donde debemos decidir.

- ¿Elijo la carrera A o la carrera B?
- ¿Me pongo este pantalón?
- Para ir al trabajo, ¿elijo el camino A o el camino B?
- Al cursar una carrera, ¿elijo el turno mañana, tarde o noche?

Por supuesto que en un problema se combinan estructuras secuenciales y condicionales.

Las **estructuras selectivas** se utilizan para tomar decisiones lógicas; de ahí que se suelen denominar también **estructuras de decisión** o

Estas estructuras de control son de gran utilidad para cuando el algoritmo a desarrollar requiera una descripción más complicada que una lista sencilla de instrucciones. Este es el caso cuando existe un número de posibles alternativas que resultan de la evaluación de una determinada condición.



En estas estructuras, se realiza una **evaluación de una condición** y de acuerdo al **resultado**, el algoritmo realiza una determinada acción. Las condiciones son especificadas utilizando expresiones lógicas.

Condicional Alternativo Simple

Este es el tipo más sencillo de estructura condicional. Sirve para implementar acciones condicionales del tipo siguiente:

- Si se verifica que una determinada **condición se cumple**, se puede ejecutar una serie de instrucciones y luego seguir adelante.
- Si la condición **NO se cumple**, NO se ejecutan dichas instrucciones y se sigue adelante.

La condición al evaluarse debe arrojar un resultado **VERDADERO** para que el conjunto de acciones pueda ejecutarse

Una vez evaluada la expresión condicional, el flujo del programa recupera su carácter secuencial y se continúa ejecutando las siguientes instrucciones.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

EJEMPLO 2

Mostrar por pantalla si un número es positivo

ACCION Ejemplo2 **ES**

AMBIENTE

a:entero

PROCESO

ESC("Ingrese el número")

LEER(a)

SI a > 0 **ENTONCES:**

ESCRIBIR("El nro. es positivo")

FIN_SI

FIN_ACCION

Condicional Alternativo Doble

Este tipo de estructura permite implementar condicionales en los que hay dos acciones alternativas:

- Si se verifica una determinada condición, ejecutar una serie de instrucciones (bloque 1).
- Si no, esto es, si la condición NO se verifica, ejecutar otra serie de instrucciones (bloque 2).

EJEMPLO 3

Mostrar por pantalla si un número ingresado es mayor o no que otro

ACCION Ejemplo3 **ES**

AMBIENTE

A,B:ENTERO

PROCESO

ESC("Ingrese el primer número"), LEER(A)

ESC("Ingrese el segundo número"), LEER(B)

SI A > B **ENTONCES**

ESC(A,"es mayor que ",B)

SINO

ESC(A,"NO es mayor que ",B)

FIN_SI

FIN_ACCION

ALGORITMOS Y ESTRUCTURAS DE DATOSUNIDAD I: Introducción a la Algoritmia

Condicional Alternativo Múltiple

Con frecuencia es necesario que existan **más de dos elecciones posibles**.

Este problema se podría resolver por estructuras selectivas simples o alternativas, anidadas o en cascada.

Pero si el número de alternativas es grande puede plantear **serios problemas de escritura** y de **legibilidad**. Por este motivo, en estos casos, se recomienda el uso de la estructura *condicional alternativo múltiple* que permite evaluar una variable que puede tomar de 1 a n valores y según ocurra uno de esos valores, se realizará una de las **n** acciones; es decir, que el programa seguirá sólo un determinado camino entre varios.

EJEMPLO 4

Dado dos números ingresados por el usuario, indique si la suma entre ellos es positiva, negativa o igual a 0.

ACCION EJEMPLO4 ES

AMBIENTE

a,b,suma: entero

PROCESO

```
ESC("Ingrese el primer número"), LEER(a)
ESC ("Ingrese el segundo número"),LEER(b)
suma := a + b
SEGÚN suma HACER
=0 : ESC("El resultado es igual a 0")
<0: ESC("El resultado es menor a 0")
>0: ESC("El resultado es mayor a 0")
FIN_SEGUN
```

FIN_ACCION



ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Es importante mencionar que la estructura anterior puede ser escrita usando los condicionales vistos anteriormente de la siguiente forma:

ACCION OtroMas ES

AMBIENTE

a,b,suma:entero

PROCESO

ESC("Ingrese el primer número"), LEER(a)

ESC ("Ingrese el segundo número"), LEER(b)

suma := a + b

SI suma = 0 ENTONCES

ESC("El resultado de la suma es 0")

SINO:

SI suma < 0 ENTONCES

ESC("El resultado de la suma es menor a 0")

SINO:

ESC("El resultado de la suma es mayor a 0")

FIN_SI

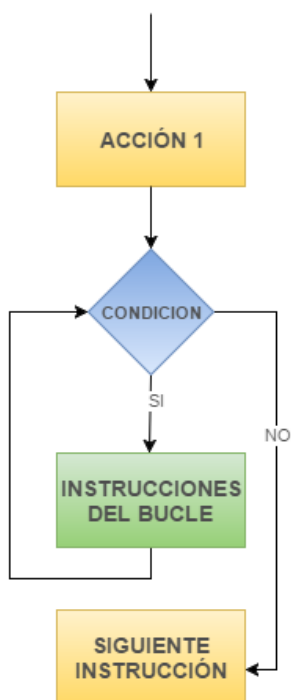
FIN_SI

FIN_ACCION

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Estructuras Repetitivas



Durante el proceso de creación de programas, es muy común, encontrarse con que una operación o conjunto de operaciones deben repetirse muchas veces. Para ello es importante conocer las estructuras de algoritmos que permiten repetir una o varias acciones, un número determinado de veces.

Las **estructuras** que repiten una secuencia de instrucciones un número determinado de veces se denominan **BUCLES**. Y cada repetición del bucle se llama **iteración**.

Todo bucle tiene que llevar **asociada una condición**, que es la que va a determinar cuándo se repite el bucle y cuando deja de repetirse.



Hay que prestar especial atención a los **bucles infinitos**, hecho que ocurre cuando la condición de finalización del bucle **no se llega a cumplir nunca**.

Estructura PRE-TEST

Esta estructura repetitiva “Pre-Test”, es en la que el cuerpo del bucle se repite siempre que se cumpla una determinada condición.

Es una **estructura repetitiva del tipo INDEFINIDA**, pues no se conoce la cantidad de veces que se debe repetir el conjunto de instrucciones del bucle.

El conjunto de acciones se ejecuta mientras la **evaluación de la condición** devuelva un resultado **verdadero**, el ciclo **se puede ejecutar 0 o más veces**.

Esto ocurre porque si inicialmente la condición no se cumple, el bucle no se ejecuta.

Pseudocódigo

```

MIENTRAS <cond> HACER
<acciones>
FIN_MIENTRAS
    
```



ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Estructura Post-Test

Esta estructura es muy similar a la anterior, sólo que, a diferencia del **PreTest**, el contenido del bucle **se ejecuta siempre al menos una vez**, ya que la evaluación de la condición se encuentra al final. De esta forma garantizamos que las acciones dentro de este bucle sean llevadas a cabo, aunque sea una vez independientemente del valor de la condición. Esta **estructura también es INDEFINIDA** y no está implementada en la mayoría de los lenguajes actuales, por lo que se reemplaza su uso con WHILE.

Pseudocódigo

```
REPETIR
    <acciones>
HASTA QUE <cond>
FIN_REPETIR
```

Estructura Manejada por contador

Esta estructura **es un Ciclo Definido**, pues se conoce de antemano la cantidad de veces que se iterará.

El **final de bucle** está controlado por un **contador** que define la cantidad de veces a iterar, por ello también se conoce como estructura Manejada por contador.

La **variable contador**, se va incrementando automáticamente de acuerdo al incremento indicado.

Si en lugar de incremento, debe ser decremento se indica con el signo "-". Si el incremento o decremento es diferente a 1, debe indicarse. Por ejemplo: Para $f := 1$ hasta 100, 2.

No hace falta inicializar la variable contador.

Pseudocódigo

```
PARA contador:=inicialización hasta fin; incremento HACER
    <acciones>
FIN_PARA
```



ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Comparación entre estructuras repetitivas

	PRE - TEST	POST-TEST	MANEJADO X CONT
Se debe conocer anticipadamente el Número de iteraciones	NO	NO	SI
En que momento se verifica la condición	Antes de la ejecución del cuerpo del bucle	Después de la ejecución del cuerpo del bucle	Antes de la ejecución del cuerpo del bucle
Puede el bucle no ejecutarse nunca	SI. Si la condición es falsa la primera vez	NO. Al menos una vez se ejecuta	SI. Si $V_f < V_i$ en FOR_TO o $V_i < V_f$ en FOR_DOWNT0
Se debe modificar el valor de la condición para finalizar el bucle?	SI, haciendo que el valor de la condición sea falsa	SI, haciendo que el valor de la condición sea verdadera	NO. Es automático
Un bucle puede ser infinito	SI	SI	NO
Cuando debe utilizarse?	El Nro de iteraciones es indeterminado y no debe ejecutarse el bucle cuando la condición es falsa la primera vez	El Nro de iteraciones es indeterminado y el bucle se debe ejecutar al menos una vez	El Nro de iteraciones se conoce por adelantado



ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Ejemplos de Aplicación

Dado un número entero N calcular la suma de todos los números entre 1 y N.

Usando Pre-Test

```
1 ACCION BUCLE1 ES
2 AMBIENTE
3     n, suma, cont: entero
4 PROCESO
5     ESCRIBIR("Ingrese el numero"), LEER(n)
6     suma := 0
7     cont := 1
8     MIENTRAS cont <= n HACER
9         suma := suma + cont
10        cont := cont + 1
11     FIN_MIENTRAS
12     ESCRIBIR("La suma es: ", suma)
13 FIN_ACCION
```

PSEUDOCÓDIGO

Usando Sublime Text

Dado un número entero N calcular la suma de todos los números entre 1 y N.

Usando Post-Test

```
1 ACCION BUCLE1 ES
2 AMBIENTE
3     n, suma, cont: entero
4 PROCESO
5     ESCRIBIR("Ingrese el numero"), LEER(n)
6     suma := 0
7     cont := 0
8     REPETIR
9         cont := cont + 1
10        suma := suma + cont
11    HASTA QUE suma >= n
12    ESCRIBIR("La suma es: ", suma)
13 FIN_ACCION
```

ALGORITMOS Y ESTRUCTURAS DE DATOSUNIDAD I: Introducción a la Algoritmia

Dado un número entero N calcular la suma de todos los números entre 1 y N.
Usando FOR

```
1  ACCION BUCLE3 ES
2  AMBIENTE
3      n, suma, cont: entero
4  PROCESO
5      ESCRIBIR("Ingrese el numero"), LEER(n)
6      suma := 0
7      PARA cont := 1 hasta n HACER
8          suma := suma + cont
9      ESCRIBIR("La suma es: ", suma)
10 FIN_ACCION
```